µOS++ III SMP — from the clone to the native test

The commands used on 2026-10-08 to clone the five repositories from `github.com/dan-maio` into `/tmp/micro-os` and run the POSIX native tests with the system compiler (`native-cmake-sys`). Section 6 lists the commands for all the other tests (native compilers, QEMU Cortex-M, SMP QEMU, real hardware).

1. Create the folder and clone

```
mkdir -p /tmp/micro-os
cd /tmp/micro-os

git clone https://github.com/dan-maio/micro-os-plus-iii.git
git clone https://github.com/dan-maio/micro-os-plus-iii-aarch32.git
git clone https://github.com/dan-maio/micro-os-plus-iii-aarch64.git
git clone https://github.com/dan-maio/micro-os-plus-iii-posix-arch.git
git clone https://github.com/dan-maio/micro-os-plus-iii-cortexm.git
```

Each clone checks out its GitHub default branch, `smp`:

Repository	Branch	Commit
`micro-os-plus-iii`	`smp`	`38076f09` docs(tests): add new-tests-diff.md and pdf comparing xpack-development …
`micro-os-plus-iii-aarch32`	`smp`	`e721dea` test(luckfox-lyra): smp_test_int4
`micro-os-plus-iii-aarch64`	`smp`	`8151c26` test(rpi3b): usb_test
`micro-os-plus-iii-posix-arch`	`smp`	`4ed05ad` chore: ignore every build* directory and the build products
`micro-os-plus-iii-cortexm`	`smp`	`71e98ff` docs: README for the six boards and the generic QEMU cores

To check what was cloned:

```
cd /tmp/micro-os
for r in micro-os-plus-iii micro-os-plus-iii-aarch32 micro-os-plus-iii-aarch64 \
         micro-os-plus-iii-posix-arch micro-os-plus-iii-cortexm; do
  echo "$r: [$(git -C $r branch --show-current)] $(git -C $r log -1 --format='%h %s')"
done
```

2. Install (xpack-dev-smp.md §13.3)

The native tests need the POSIX-arch port, so only that port is installed and linked; then the kernel test harness installs its dependencies.

```
cd /tmp/micro-os/micro-os-plus-iii-posix-arch
xpm install
xpm link

cd /tmp/micro-os/micro-os-plus-iii/tests
xpm run install-all
```

Result: both returned 0.

The kernel harness finds the ports as siblings of `micro-os-plus-iii` (`tests/cmake/tests-main.cmake`), which is why all five clones sit side by side in `/tmp/micro-os`.

3. Run the native tests (system compiler)

```
cd /tmp/micro-os/micro-os-plus-iii/tests
xpm run test-native-cmake-sys
```

`test-native-cmake-sys` runs, for the `debug` and then the `release` configuration:

```
xpm run prepare --config native-cmake-sys-debug
xpm run build   --config native-cmake-sys-debug
xpm run test    --config native-cmake-sys-debug
xpm run prepare --config native-cmake-sys-release
xpm run build   --config native-cmake-sys-release
xpm run test    --config native-cmake-sys-release
```

The compiler is the host's (`gcc (GCC) 16.2.1`); `test` runs `ctest -V` in `build/<config>`.

4. Results

Configuration	Result
`native-cmake-sys-debug`	16/16 passed
`native-cmake-sys-release`	16/16 passed

`xpm run test-native-cmake-sys` returned 0.

The 16 tests: flatfs-test, mutex-ceiling-test, mutex-stress, rtos-apis, smp-mat-test, smp-mutex-stress, smp-num-test, smp-pipeline-test, smp-pro-cons-test, smp-rtos-apis, smp_test0 … smp_test4, cmsis-os-validator (each `native-<name>-host`).

To re-run a single test:

```
cd /tmp/micro-os/micro-os-plus-iii/tests/build/native-cmake-sys-debug
ctest -R '^native-smp-pipeline-test-host$' --output-on-failure
```

5. Possible correction: `env` must be `/usr/bin/env`

`micro-os-plus-iii/test_smpl/run-host.sh` starts each test binary through `env`, found in `PATH`:

```
timeout "$tmo" env "${env[@]}" "$exe" 2>&1 | tee "$log"    # line 78
timeout "$tmo" env "${env[@]}" "$exe" > "$log" 2>&1        # line 81
```

On some systems `PATH` puts `~/.local/bin` before `/usr/bin`, and `~/.local/bin` holds another program named `env`. That one runs instead of `/usr/bin/env`, and the test binaries are not started: every port test (`native-*-host`) prints nothing, ends in 0.02 s and is reported as `NO RESULT (rc=0)`, while `native-cmsis-os-validator-host` (started directly by `ctest`) passes.

Check which `env` is used:

```
type -a env
```

The `env` used must be `/usr/bin/env`. The correction, in `test_smpl/run-host.sh`, is to call it by its full path on both lines:

```
timeout "$tmo" /usr/bin/env "${env[@]}" "$exe" 2>&1 | tee "$log"
timeout "$tmo" /usr/bin/env "${env[@]}" "$exe" > "$log" 2>&1
```

6. All the other tests

Every command below is an action of `micro-os-plus-iii/tests/package.json` and runs in `micro-os-plus-iii/tests`.

6.1 Install for all the tests (xpack-dev-smp.md §13.3)

The native tests need only posix-arch (section 2). The QEMU Cortex-M, SMP and hardware tests also need the other three ports:

```
cd /tmp/micro-os/micro-os-plus-iii-posix-arch && xpm install && xpm link
cd /tmp/micro-os/micro-os-plus-iii-cortexm    && xpm install && xpm link
cd /tmp/micro-os/micro-os-plus-iii-aarch32    && xpm install && xpm link
cd /tmp/micro-os/micro-os-plus-iii-aarch64    && xpm install && xpm link

cd /tmp/micro-os/micro-os-plus-iii/tests
xpm run install-all        # npm install ; xpm install --all-configs
```

`install-all` installs the toolchains of every configuration (xPack GCC, clang, arm-none-eabi-gcc, aarch64-none-elf-gcc, qemu-arm, openocd).

6.2 Everything at once

```
xpm run test-all           # test-native-cmake ; test-cortex-cmake ; test-smp-cmake
```

An action stops at its first failing test; to continue with the next group, run the groups one by one (6.3–6.5).

6.3 Native (POSIX host)

Action	Compiler	Configurations (debug + release)
`xpm run test-native-cmake-sys`	the host's gcc	`native-cmake-sys-*`
`xpm run test-native-cmake-gcc`	xPack gcc 14.2.0	`native-cmake-gcc-*`
`xpm run test-native-cmake-gcc11`	xPack gcc 11.5.0	`native-cmake-gcc11-*`
`xpm run test-native-cmake-gcc12`	xPack gcc 12.4.0	`native-cmake-gcc12-*`
`xpm run test-native-cmake-gcc13`	xPack gcc 13.3.0	`native-cmake-gcc13-*`
`xpm run test-native-cmake-gcc14`	xPack gcc 14.2.0	`native-cmake-gcc14-*`
`xpm run test-native-cmake-clang`	xPack clang 19.1.7	`native-cmake-clang-*`
`xpm run test-native-cmake-clang13` … `clang19`	xPack clang 13.0.1 … 19.1.7	`native-cmake-clang13-*` … `clang19-*`

`xpm run test-native-cmake` runs `gcc11`, `gcc12`, `gcc13` and `gcc14` (not on macOS, which runs `test-native-cmake-sys` instead), then `clang16`, `clang17`, `clang18` and `clang19`; it only prints the `clang13`–`clang15` commands.

The `test` step runs `ctest -V`, except `native-cmake-gcc-*`, which runs `ctest -V -LE hwd`.

6.4 QEMU Cortex-M (single core, xPack arm-none-eabi-gcc 15.2.1)

The QEMU these tests run is the newest `qemu-arm` xPack installed in `~/.local/xPacks/@xpack-dev-tools/qemu-arm/` (9.2.4-1.1 on 2026-10-09), found by the platform's `CMakeLists.txt`; the 8.2.6 that `xpm install` installs for these configurations is used only when no newer one is there.

```
xpm run test-cortex-cmake          # the four below
xpm run test-qemu-cortex-m0-cmake
xpm run test-qemu-cortex-m3-cmake
xpm run test-qemu-cortex-m4f-cmake
xpm run test-qemu-cortex-m7f-cmake
```

`xpm run run-qemu-cortex-latest` runs the same four; `xpm run test` runs only `test-qemu-cortex-m7f-cmake`.

6.5 SMP on QEMU (xPack arm-none-eabi-gcc / aarch64-none-elf-gcc 15.2.1)

```
xpm run test-smp-cmake             # the eight below
xpm run test-aarch32-rpi-zero-2w-cmake
xpm run test-aarch32-rpi3b-cmake
xpm run test-aarch64-rpi-zero-2w-cmake
xpm run test-aarch64-rpi3b-cmake
xpm run test-2xcortex-m33-cmake
xpm run test-pico2-1cpu-cmake
xpm run test-cortexm-pico2-cmake
xpm run test-cortexm-pico2-rp2350b-psram-cmake
```

The `test` step of these configurations runs `ctest -V -LE hwd`: the real-hardware tests are built but not run.

6.6 One configuration, step by step

Every test action above is `prepare`, `build` and `test` for the `-debug` and then the `-release` configuration. One configuration alone:

```
xpm run prepare --config <configuration>
xpm run build   --config <configuration>
xpm run test    --config <configuration>
```

For example `<configuration>` = `aarch32-rpi3b-cmake-gcc-debug`. One test of a configuration that is already built:

```
cd build/<configuration>
ctest -R '^<test name>$' --output-on-failure
ctest -N                  # lists the test names
```

6.7 Real hardware (`hwd`): board and debug probe connected

These run one test on the board:

```
xpm run test-<name>-hwd --config <configuration>
```

Each one runs the CMake prepare with the toolchain, the build, then `ctest -V -R <platform>-<name>-hwd` in `build/<configuration>`. For example:

```
xpm run test-mutex-stress-hwd --config aarch32-rpi3b-cmake-gcc-debug
xpm run test-smp-test0-hwd    --config cortexm-pico2-cmake-gcc-debug
xpm run test-sd_test-hwd      --config aarch32-luckfox-lyra-cmake-gcc-debug
```

The <name> values, per configuration (-debug and -release alike):

Configurations	test-<name>-hwd actions
aarch32-rpi-zero-2w-*, aarch64-rpi-zero-2w-*	sd_test, smp-mat-sdcard-test, smp-mat-test, smp-num-test, smp-pipeline-test, smp-pro-cons-test, smp_test0 … smp_test4, usb_test, mutex-stress, rtos-apis, cmsis-os-validator
aarch32-rpi3b-*, aarch64-rpi3b-*	the same, without usb_test
aarch32-luckfox-lyra-*	sd_test, smp-mat-sdcard-test, smp-mat-test, smp-num-test, smp-pipeline-test, smp-pro-cons-test, smp_test0 … smp_test7, smp_test_int, smp_test_int2 … smp_test_int5, mutex-stress-test
cortexm-pico2-*	clock-test, cmsis-os-validator, cmsis-os-validator-ram, exc-test, mutex-stress, mutex-stress-ram, fp-switch, rtos-apis, rtos-apis-ram, sc-test-ko, smp-mat-test, smp-mat-test-ram, smp-test-ko, smp-test-usb-cdc-acm, smp-test-usb-hid, smp-test0 … smp-test5
cortexm-pico-* (RP2040)	the same as cortexm-pico2-*
cortexm-pico2-pizero-*	exc-test, sc-test-ko, smp-mat-test, smp-test-ko, psram-exec, psram-mat-test-250, smp-test-usb-cdc-acm, smp-test-usb-hid, smp-test0 … smp-test5
cortexm-pico2-rp2350b-psram-*	exc-test, fp-switch, sc-test-ko, smp-mat-test, smp-test0 … smp-test5, smp-test-ko, smp-test-nested, smp-test-nested-clock, smp-test-nested-clock_200, smp-test-nested-clock_250
cortexm-nucleof411-*	cmsis-os-validator, mos-test1, mutex-stress, rtos-apis
cortexm-weactf411-*	the same, plus spi-pipeline
cortexm-weactf412-*	the same as nucleof411, plus uart-test1

cortexm-pico2-pizero, cortexm-nucleof411, cortexm-weactf411, cortexm-weactf412 and aarch32-luckfox-lyra have no test-<platform>-cmake action; they are built with prepare/build --config (6.6).

Raspberry Pi (Zero 2 W, 3 B), both ports. Each test resets the board through its watchdog, loads the image, starts **core 0 only**, and resumes cores 1–3 where the firmware parked them; the test's kernel releases them itself (UOS_HW_RESUME=cpsr-first / pc-first, no __smp_spin writes). This needs an SD card that boots the port's width (arm_64bit=0 for aarch32-*, arm_64bit=1 for aarch64-*) with a kernel that leaves cores 1–3 in the firmware's loop. Tested on a Pi Zero 2 W on 2026-10-09, all tests but usb_test, on both ports. The commands, one by one: RPI-ZERO-2W-HARDWARE-STEPS.md; the previous flow is kept in each port's test/boards/rpi-zero-2w/hw.sh.bak.

6.8 Legacy single-core boards

```
xpm run test-raspberrypi-pico-cmake
xpm run test-nucleo-f411re-cmake
xpm run test-nucleo-f767zi-cmake
xpm run test-nucleo-h743zi-cmake
```

xpack-dev-smp.md §21.3 records that these four fail at configure (the cortexm CMake does not find the kernel through the xpacks path).

6.9 Results recorded on 2026-10-07 (xpack-dev-smp.md §23.4)

Action	Debug	Release
`test-native-cmake-sys`	16/16	16/16
`test-cortex-cmake` (m0, m3, m4f, m7f)	3/3 each	3/3 each
`test-aarch32-rpi-zero-2w-cmake`, `test-aarch32-rpi3b-cmake`	15/15	15/15
`test-aarch64-rpi-zero-2w-cmake`, `test-aarch64-rpi3b-cmake`	15/15	15/15
`test-2xcortex-m33-cmake`	4/4	4/4
`test-pico2-1cpu-cmake`	4/4	4/4
`test-cortexm-pico2-cmake`	6/6	6/6
`test-cortexm-pico2-rp2350b-psram-cmake`	3/3	3/3

The other native compilers (6.3) and the hardware tests (6.7) were not run for this document.